

PROJET D'ÉTUDES — RENDU INDIVIDUEL

ToolboxV8

Plateforme d'automatisation de tests d'intrusion

Auteur	Ayoub BEN RACHED
Rôle principal	Architecte & Back-end / Sécurité — split api/web, orchestration Celery, JWT/RBAC/Fernet, HTTPS Caddy, SIEM
Promotion	Mastère Cybersécurité 2025/2026
École	SUP DE VINCI
Date	Juin 2026

1. Perspectives d'évolution

1.1 Boucler la chaîne de réponse automatique (SOAR)

Aujourd'hui, le module de réponse défensive (`block_ip` , `isolate_host` , `send_alert`) est fonctionnel mais déclenché manuellement. La suite logique serait d'exposer une API REST (`POST /api/defensive/response/block-ip`) et de la brancher à une règle de corrélation Elasticsearch : une alerte Snort `sqlmap` ou un brute-force SSH déclencherait alors automatiquement le blocage de l'IP source. On passerait d'une réponse semi-manuelle à un véritable mini-SOAR, ce qui correspond aux attentes d'un SOC moderne.

1.2 Déployer l'IDS Snort en production

Les neuf règles Snort et le pipeline Logstash sont écrits et prêts, mais le conteneur n'est pas déployé à cause des contraintes de mode promiscuous sous Docker Desktop Windows/WSL2. Sur un hôte Linux ou une VM Kali dédiée, l'activation est rapide et débloquerait la détection IDS de bout en bout.

1.3 Gestion des secrets par coffre-fort

Les secrets (`SECRET_KEY` , `FERNET_KEY` , mots de passe) transitent aujourd'hui par un fichier `.env` . En production, je migrerais vers un coffre-fort (HashiCorp Vault ou équivalent) avec rotation

automatique, ce qui éliminerait le risque de fuite par fichier et professionnaliserait la gestion du cycle de vie des clés.

1.4 Industrialisation du déploiement

La CI construit déjà les images mais ne les pousse pas encore sur un registry. L'étape suivante : publication versionnée des images, puis orchestration (Kubernetes) pour passer d'un `docker-compose` mono-machine à un déploiement scalable et résilient, en cohérence avec une logique PRA/PCA.

1.5 Authentification d'entreprise (SSO / MFA)

L'authentification JWT actuelle est solide mais isolée. Une intégration SSO via OIDC, complétée par du MFA, permettrait d'insérer la toolbox dans l'écosystème d'identité d'un client sans gérer de comptes séparés. Un vrai plus pour l'adoption en entreprise.

2. Analyse critique — limites et retours d'expérience

2.1 Le token en localStorage, une dette que j'aurais pu éviter

Au départ, j'avais stocké le JWT dans le `localStorage` du navigateur. C'est simple, mais c'est exactement le genre de choix qui expose une application au vol de session par XSS. Paradoxal pour un outil de sécurité. J'ai dû refondre toute l'authentification en **cookie HttpOnly** et séparer l'application en deux services (`api` et `web`). Avec le recul, j'aurais dû partir directement sur cette architecture : la refonte a coûté du temps qu'une bonne décision initiale aurait économisé.

2.2 Un bug d'authentification qui n'en était pas un

Pendant un moment, l'authentification renvoyait « identifiants incorrects » alors que les identifiants étaient bons. J'ai cherché longtemps du côté des mots de passe avant de comprendre que le `TrustedHostMiddleware` rejetait l'hôte interne `api:8000`. **Leçon** : sur une architecture multi-services, la cause d'une erreur n'est pas toujours là où le symptôme apparaît. Il faut raisonner sur toute la chaîne.

2.3 Le build Kali et le verrou PEP 668

Installer les dépendances Python sur l'image Kali Rolling échouait à cause du verrouillage d'environnement (PEP 668). J'ai conçu un build **multi-stage** (export des dépendances dans une image slim, puis installation dans un venv côté Kali). Ça m'a obligé à comprendre finement le fonctionnement des images Docker, mais c'est une compétence que je retiens.

2.4 Caddy et l'erreur TLS au premier démarrage

Mon Caddyfile initial déclarait `:443` sans nom d'hôte, ce qui empêchait Caddy de générer le bon certificat serveur (`ERR_SSL_PROTOCOL_ERROR`). Le correctif, qui consistait à passer à `localhost`, était simple, mais le diagnostic m'a appris à comprendre le fonctionnement interne d'un outil avant de l'intégrer, plutôt que de le traiter comme une boîte noire.

2.5 Une réponse défensive restée semi-automatique

Le `ResponseModule` fonctionne, mais son déclenchement reste manuel. Faute de temps, je n'ai pas bouclé la chaîne SOAR complète (cf. §1.1). C'est la limite que je regrette le plus, car elle aurait été la démonstration la plus forte du volet défensif.

3. Analyse personnelle

3.1 Ce que ce projet m'a vraiment appris

Mon rôle d'architecte m'a fait prendre conscience que mes décisions techniques avaient un **impact direct sur le travail des deux autres** : Abdallah intégrait ses outils et Titouan construisait l'interface sur la fondation que je livrais. Cette responsabilité m'a poussé à privilégier la stabilité et la cohérence plutôt que mes préférences personnelles. J'ai aussi appris à **arbitrer entre l'idéal et le réalisable** par exemple assumer que Snort ne serait pas déployé plutôt que de bloquer tout le projet sur une contrainte d'environnement.

Surtout, j'ai intériorisé le principe de **sécurité par conception** : sur un outil offensif, le moindre relâchement (un token mal stocké, un secret en clair) devient une vulnérabilité critique. C'est une exigence que je n'avais jamais ressentie aussi concrètement.

3.2 L'utilisation de l'IA dans le projet

J'ai utilisé des outils d'IA comme **assistant de développement**, principalement pour accélérer l'écriture de code répétitif (configurations Docker, squelettes de routes FastAPI), pour débloquer des erreurs d'intégration et pour relire ma documentation technique. Mon constat rejoint celui de l'équipe : l'IA est un bon accélérateur **sur ce qu'on maîtrise déjà**, mais elle peut induire en erreur sur ce qu'on connaît mal, car elle n'a pas le contexte de notre architecture (elle ne pouvait pas, par exemple, deviner les contraintes de notre découpage api/web ou du build Kali). Je l'ai donc utilisée comme un outil à valider systématiquement, jamais comme une source de vérité.

3.3 Forces

- **Vision d'architecture** : capacité à concevoir un découpage (api/web) qui résout un vrai problème de sécurité tout en gardant l'API ouverte aux intégrations.
- **Rigueur sécurité** : application systématique des bonnes pratiques (HttpOnly, RBAC, Fernet, HTTPS, audit) à notre propre outil.
- **Ténacité au débogage** : diagnostic d'erreurs distribuées non triviales (middleware, TLS, PEP 668).
- **Documentation argumentée** : justification transparente de mes choix techniques, y compris des limites assumées.

3.4 Faiblesses

- **Refonte évitable** : être parti sur `localStorage` avant de basculer en cookie HttpOnly a généré une dette technique.
-

- **Tendance à vouloir tout sécuriser parfaitement** : j'ai parfois passé du temps sur le durcissement avant que les fonctionnalités de base soient stabilisées.
- **Automatisation défensive inachevée** : la chaîne SOAR aurait pu être priorisée plus tôt.

3.5 Compétences développées

- **Architecture sécurisée** : découpage de services, cloisonnement réseau, principe de moindre exposition.
- **IAM / Management de la sécurité** : JWT, cookie HttpOnly, RBAC multi-rôles.
- **Cryptologie appliquée** : chiffrement Fernet, hachage bcrypt, gestion des clés.
- **DevSecOps / PRA-PCA** : Docker multi-stage, Docker Compose, CI/CD (GitHub Actions + GitLab CI), reverse proxy HTTPS.
- **SIEM & défense** : déploiement ELK, indexation d'événements, module de réponse active.
- **Diagnostic de systèmes distribués** : raisonner sur l'ensemble d'une chaîne multi-services.

3.6 Pour les prochains projets

- **Choisir l'architecture sécurisée dès le départ** plutôt que de refactorer (le cas du cookie HttpOnly).
- **Prioriser la boucle complète** d'une fonctionnalité (détection → réponse automatique) avant de la peaufiner.
- **Documenter les décisions d'architecture au fil de l'eau**, pas seulement le code.
- **Industrialiser tôt** la gestion des secrets et le déploiement, plutôt que de repousser ces sujets en fin de projet.

Document rédigé dans le cadre du Projet d'Études — Mastère Cybersécurité 2025/2026 — SUP DE VINCI